

file == uart\_polished.v

// create a module, it does stuff once you instantiate it in uart\_tb\_polished.v

```

1) module uart_tx(
    input ...,
    output ...,
    ...
);

```

// this stuff (lets you know when ur done sending bits & to go to new state, store bits)

2) internal stuff we do not want in our simulation file

Ex. reg [2:0] bit\_counter;

3) always @ (posedge clk)

// on posedge, this forever loop runs based on inputs defined in your tb

note: inputs are subject to change

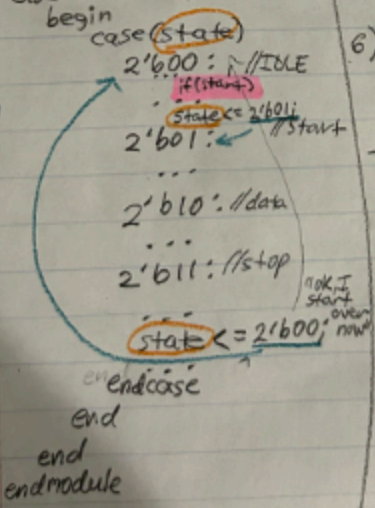
```

begin
    if (reset)
        state <= 2'b00;
        busy ...
        tx ...
        bit_counter ...
        display_reg ...
    end
    else
        begin
            case (state)

```

// recall FSM?

IDLES  
START  
DATA  
STOP



uart\_tb\_polished.v == file (note, v = verilog)

1) timescale 1ns/1ps // what unit of time / precision

2) module uart\_tx\_tb;

3) reg clk;  
reg reset;  
...  
wire tx;

// DO NOT include the internal stuff mentioned in other file here, we do not want to see that

4) uart\_tx uut ( //uart\_tx look familiar? we create copy here  
 .clk(clk),  
 .tx(tx),  
 ...  
);

5) always // loop

6) begin // clock cycle every 10 units of time  
 #5 clk = ~clk;  
end

7) initial // start when sim. starts

8) begin  
 start = 0; // define inputs here  
 reset = 1;  
 clk = 0;  
 data = 8'b10101101;  
 \$dumpfile("bananas.vcd");  
 // file that store GTrace output  
 // proven name is arbitrary  
 \$dumpvars(0, uart\_tx\_tb);

#10;  
reset = 0; // allows us to escape if (reset) in other file  
#10;  
start = 1; // allows access to (start)  
#10;  
start = 0; // allows us to move to new state & start over  
#120  
\$finish; // enough time for sim.